Once we have our updated node matrix, we can reduce it down to a single vector by either averaging or max-pooling over the node dimension to get a single *d*-dimensional vector that should summarize the graph as a whole. We can pass that through a few ordinary linear layers before getting our final output, which is just a vector of Q values. Thus we are building a relational deep Q-network (Rel-DQN).

10.3.4. Training the relational module

You might have noticed the last function call in the code is actually `log_softmax`, which is not something we would use for Q-learning. But before we get to Q-learning, we will test our relational module on classifying MNIST digits and compare it to a conventional nonrelational convolutional neural network. Given that our relational module has the ability to model long-distance relationships in a way that a simple convolutional neural network cannot, we would expect our relational module to perform better in the face of strong transformations. Let's see how it does.

Listing 10.4. MNIST training loop

```
agent = RelationalModule()                                          1
epochs = 1000
batch_size=300
lr = 1e-3
opt = torch.optim.Adam(params=agent.parameters(),lr=lr)
lossfn = nn.NLLLoss()
for i in range(epochs):
    opt.zero_grad()
    batch_ids = np.random.randint(0,60000,size=batch_size)          2
    xt = mnist_data.train_data[batch_ids].detach()
    xt = prepare_images(xt,rot=30).unsqueeze(dim=1)                 3
    yt = mnist_data.train_labels[batch_ids].detach()
    pred = agent(xt)
    pred_labels = torch.argmax(pred,dim=1)                          4
    acc_ = 100.0 * (pred_labels == yt).sum() / batch_size           5
    correct = torch.zeros(batch_size,10)
    rows = torch.arange(batch_size).long()
    correct[[rows,yt.detach().long()]] = 1.
    loss = lossfn(pred,yt)
    loss.backward()
    opt.step()
```